

DFS와 BFS의 특징 및 장단점 비교

1. DFS (Depth-First Search, 깊이 우선 탐색)

특징 및 장점: DFS는 스택 자료구조 또는 재귀 호출을 이용하여 구현되며, 한 경로를 끝까지 탐색한 후 다른 경로를 탐색하는 방식이다. 이러한 특성으로 인해 메모리 사용량이 상대적으로 적다는 장점이 있다. 현재 탐색 중인 경로상의 노드들만 스택에 저장하면 되기 때문에, 그래프의 분기가 많더라도 메모리 효율성이 높다.

또한 목표 노드가 트리의 깊은 레벨에 위치할 경우 BFS보다 빠르게 해당 노드를 발견할 수 있다. 재귀적 구현이 가능하여 코드의 간결성과 가독성이 우수하며, 백트래킹 알고리즘이나 위상 정렬 등에서 효과적으로 활용된다.

단점: DFS의 가장 큰 단점은 최단 경로를 보장하지 못한다는 점이다. 먼저 발견된 경로가 반드시 최단 경로가 아니므로, 최단 거리를 구해야 하는 문제에는 적합하지 않다. 또한 깊이 제한을 설정하지 않으면 무한 루프에 빠질 위험이 있으며, 목표 노드가 얕은 레벨에 있더라도 불필요하게 깊은 탐색을 수행할 수 있어 비효율적일 수 있다.

2. BFS (Breadth-First Search, 너비 우선 탐색)

특징 및 장점: BFS는 큐 자료구조를 이용하여 구현되며, 시작 노드로부터 가까운 노드부터 레벨 순서대로 탐색하는 방식이다. 이러한 특성 덕분에 가중치가 없는 그래프에서 최단 경로를 보장한다는 중요한 장점이 있다. 처음 목표 노드를 발견했을 때가 곧 최단 경로이므로, 네비게이션 시스템이나 최단 거리 문제에 적합하다.

또한 레벨별로 체계적인 탐색이 가능하여 목표 노드가 얕은 레벨에 있을 경우 효율적으로 찾을 수 있으며, 무한 루프에 빠질 위험이 DFS에 비해 적다.

단점: BFS의 주요 단점은 메모리 사용량이 많다는 것이다. 큐에 현재 레벨의 모든 노드를 저장해야 하므로, 그래프의 분기 계수가 클 경우 메모리 부담이 기하급수적으로 증가한다. 특히 목표 노드가 깊은 레벨에 위치할 경우, 그 이전의 모든 레벨을 탐색해야 하므로 시간과 공간 복잡도 측면에서 비효율적일 수 있다.

3. 결론

DFS와 BFS는 시간 복잡도는 $O(V+E)$ 로 동일하지만 (V: 정점 수, E: 간선 수), 공간 복잡도와 특성이 다르다. 메모리 제약이 있거나 해의 존재 여부만 확인하면 되는 경우 DFS가 유리하며, 최단 경로를 구하거나 레벨별 분석이 필요한 경우 BFS가 적합하다. 따라서 그래프의 구조, 메모리 제약, 목표 노드의 위치 등을 종합적으로 고려하여 최적의 알고리즘을 선택해야 한다.